PRO GIT & GITHUB ARCHITECTURE SERIES • STAFF ENGINEER REVIEWED

The Software Engineer's Complete Git & GitHub CLI Handbook

Every concept anchored to your Java project (**Main.java** & **Calculator.java**) with exact **Before** → **Command** → **After** diffs, **.git** internals, and visual commit graphs.

[DEBATE] How Multi-Agent Debate Upgraded This Handbook (Lead Author vs. Staff Engineer Critic)

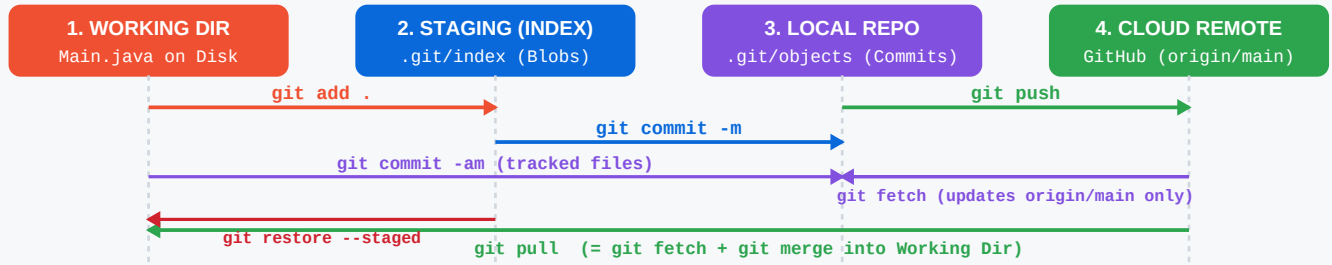
Before generating this PDF, **Agent 1 (Lead Author)** and **Agent 2 (git-staff-critic)** conducted a two-round architectural critique. The Staff Critic rejected a generic tutorial outline and mandated **7 production-engineering upgrades**: (1) Anchoring all 8 modules to your real **github-revision** Java repo & commit SHAs (**03391fe** → **8257653** → **7be836b** → **442b5b5** → **e81ae14**); (2) Explaining why uncommitted edits travel across **git switch** & when they block switching; (3) Exposing the **--ours / --theirs inversion trap** during **git rebase** vs **git merge**; (4) Adding the **3-Tree State Matrix** for **git reset (--soft, --mixed, --hard)** + **git reflog** rescue; (5) Contrasting **git fetch** vs **git pull** vs **git pull --rebase**; (6) Mandating **--force-with-lease** over **--force**; and (7) Adding a 10-scenario **'Oh Shit, Git!' Emergency Rescue Playbook**.

01 Git Architecture, The 4 Zones & Diffing

Working Directory ↔ Staging Area (Index) ↔ Local Repo (.git) ↔ Cloud Remote (origin)

Most Git confusion disappears once you see Git as moving snapshots across **4 distinct zones**. Every CLI command you ran today on **Main.java** and **Calculator.java** moves data between specific zones:

ILLUSTRATION 1: GIT'S 4 ZONES & EXACT DATA-FLOW COMMANDS



COMMAND EXECUTED: `git add . && git commit -m 'Add Calculator' && git push`

BEFORE: Calculator.java added, Main.java modified (Unstaged)

```
Working Dir : Main.java (modified), Calculator.java (untracked)
Staging Area: [Empty - matches 03391fe]
Local Repo : 03391fe (Initial commit)
! Plain 'git diff' ONLY shows Main.java (already tracked)!
```

AFTER: Snapshot 8257653 saved locally & synced to origin/main

```
+ Staging Area: Hashes both files into Blobs (035582b, d8ead7c)
+ Local Repo : Creates Commit 8257653 pointing to new Tree
+ Cloud Remote: Uploads 8257653 -> origin/main matches HEAD
+ Working Dir : Clean (matches HEAD)
```

CLI Command	Zones Compared / Affected	What You See in Your Java Project
`git diff`	Working Dir vs. Staging Area	Unstaged edits in **already-tracked** files (**Main.java**). Untracked files don't show yet!
`git diff --staged`	Staging Area vs. Local HEAD	Exact lines staged for the next commit (includes newly added **Calculator.java**).
`git diff origin/main..HEAD`	Cloud Remote vs. Local HEAD	Everything you committed locally that hasn't been pushed to GitHub yet.
`git fetch origin`	Cloud → .git/refs/remotes/origin	Downloads new cloud commits & updates **origin/main** pointer, without touching your files.
`git pull (vs --rebase)`	Cloud → Local Repo + Working Dir	Runs **git fetch** + **git merge origin/main** (or replays cleanly with **--rebase**).

02 Under the Hood: .git Internals, Blobs, Trees & Ignore Rules

How Git stores snapshots in .git/objects and why Blobs never store filenames

When you ran **git add .** and **git commit** on your Java project, Git stored a complete content-addressable snapshot in **.git/objects/** using SHA-1 hashes (first 2 hex characters = folder name, remaining 38 = filename):

ILLUSTRATION 2: INSIDE .git/objects — CONTENT-ADDRESSABLE GRAPH OF YOUR COMMIT (8257653)



● ● ● Terminal — Inspecting Your Real Commit (8257653) Inside .git/objects

```
$ git cat-file -t 8257653 # Returns object type: commit, tree, or blob
commit
$ git cat-file -p 8257653 # Prints commit metadata + root tree SHA + parent SHA
tree 2cb657ca99b78727bc943a26a400a365fe9582d8
parent 03391fe8e03dfdc65d28b0bf5614e5a1c9bd0ded
$ git ls-tree 8257653 src/ # Prints Tree table mapping permissions + filenames -> Blob SHAs
100644 blob 035582b9e1f4e2f875579d50e44eb0c9ff6dda34 src/Calculator.java
100644 blob d8ead7c122233f6254ed2bde827e0d047a486b36 src/Main.java
$ git cat-file -p 035582b # Prints raw file content of Calculator.java (no filename stored!)
+public class Calculator { public static int add(int a, int b) { return a + b; } }
```

[IMPORTANT] The .gitignore Trap & .gitignore vs .git/info/exclude

- **.gitignore** is committed and shared with the whole team (e.g., **out/**, ***.class**, **.DS_Store**).
- **.git/info/exclude** is local-only on your machine (not pushed to GitHub)—ideal for personal scratch files.
- **Production Trap:** Adding **out/Main.class** to **.gitignore** does nothing if that file was already committed earlier! To stop tracking an already-committed file without deleting it from disk, run: **git rm --cached out/Main.class** and commit.

03 Branching, Merging & Merge Conflict Anatomy

Why uncommitted edits travel across git switch, Fast-Forward vs 3-Way Merge, and Conflict Resolution

[TIP] Why Didn't subtract() Disappear When You Switched Branches Before Committing?

Earlier today, when we added **subtract()** to **Calculator.java** on **feat/subtract** and you ran **git switch main** before running **git commit**, the method stayed visible on **main**! Why? Because uncommitted changes live in your **Working Directory / Index**—they do not belong to any branch until committed. Once you committed **7be836b** on **feat/subtract**, switching to **main** immediately replaced the working files with **8257653**.

COMMAND EXECUTED: `git merge feat/multiply --> git checkout --theirs src/Main.java && git commit`

BEFORE: Diverged Line 3 in src/Main.java Across Two Branches

```
Branch main (commit 0f82c1a):
- System.out.println("Welcome v1.0 (Main Branch)!");
Branch feat/multiply (commit e81ae14):
+ System.out.println("Welcome v2.0 (Multiply
Branch)!");
Both modified Line 3 starting from base 442b5b5!
```

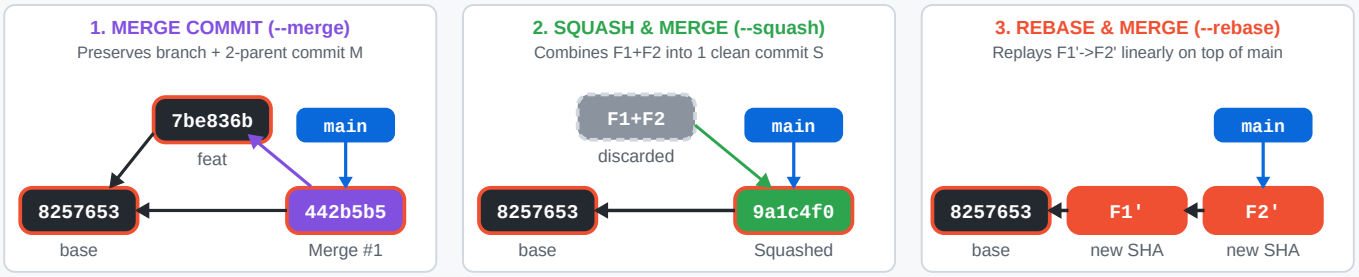
AFTER: Conflict Markers Injected -> Resolved Cleanly

```
<<<<<<< HEAD (Current Change / --ours)
  System.out.println("Welcome v1.0 (Main Branch)!");
=======
  System.out.println("Welcome v2.0 (Multiply
Branch)!");
>>>>>>> feat/multiply (Incoming Change / --theirs)
+ Resolved with --theirs -> keeps v2.0 and creates
merge commit!
```

04 Pull Requests, GitHub CLI (gh) & The 3 Merge Strategies

Opening, reviewing, and merging PRs from the terminal + deleting cloud vs local branches

ILLUSTRATION 3: THE 3 GITHUB PULL REQUEST MERGE STRATEGIES (COMMIT GRAPH COMPARISON)



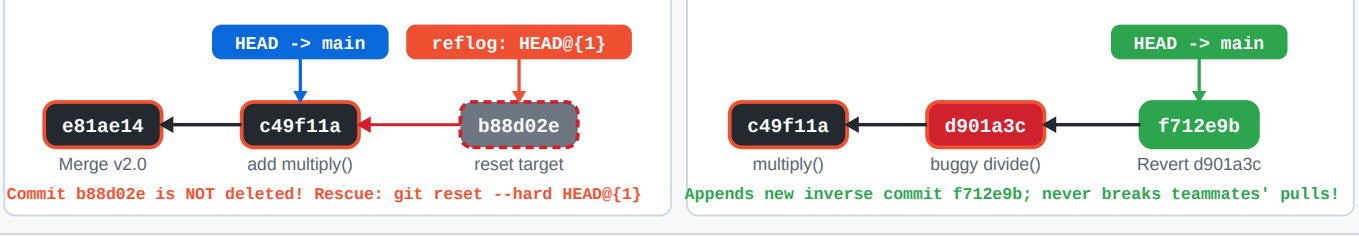
● ● ● Complete 100% CLI Pull Request Lifecycle (What You Ran on feat/subtract)

```
# 1. Push feature branch to GitHub cloud and set upstream tracking (-u)
$ git push -u origin feat/subtract
# 2. Open Pull Request targeting main directly from terminal
$ gh pr create --base main --head feat/subtract --title "Add subtract" --body "Adds Calculator.subtract"
# 3. Inspect PR metadata, view color diff in CLI, and leave a review comment
$ gh pr view 1 && gh pr diff 1 && gh pr review 1 --comment --body "LGTM!"
# 4. Merge PR on GitHub (--merge, --squash, or --rebase) and sync local main
$ gh pr merge 1 --merge --delete-branch && git switch main && git pull
# 5. Delete local branch vs. Delete remote cloud branch
$ git branch -d feat/subtract # Deletes LOCAL branch only
$ git push origin --delete feat/subtract # Deletes CLOUD branch on GitHub (+ git fetch --prune)
```

05 Time Travel & Undoing Mistakes: reset, revert, Detached HEAD & reflog

The 3-Tree Matrix of git reset, safe cloud undo with git revert, and rescuing lost commits

ILLUSTRATION 4: git reset --hard (+ git reflog RESCUE) VS. git revert (SAFE FOR SHARED MAIN)



Command	HEAD & Branch	Staging (Index)	Working Dir	When Engineers Use It
`git reset --soft HEAD~1`	Back 1 commit	KEPT STAGED	KEPT INTACT	Undo last commit but keep changes staged to recommit or squash.
`git reset --mixed HEAD~1`	Back 1 commit	UNSTAGED	KEPT INTACT	Default reset: unstages changes so you can edit/re-add selectively.
`git reset --hard HEAD~1`	Back 1 commit	WIPED	WIPED	Discard the last commit AND uncommitted changes locally.
`git revert <commit-sha>`	FORWARD (+1)	Clean	Inverse diff	**SAFE FOR CLOUD:** Undoes a pushed commit on shared **main** safely.

COMMAND EXECUTED: `git reflog --> git reset --hard c49f11a (or HEAD@{1})`

BEFORE: Added multiply() in Commit c49f11a, Then Ran git reset --hard HEAD~1

```
$ git reset --hard 442b5b5
- multiply() vanishes from Calculator.java!
- git log no longer shows commit c49f11a!
! Did you lose your code forever? NO — Git keeps orphaned commits ~90 days.
```

AFTER: Rescued! multiply() Restored Instantly via git reflog

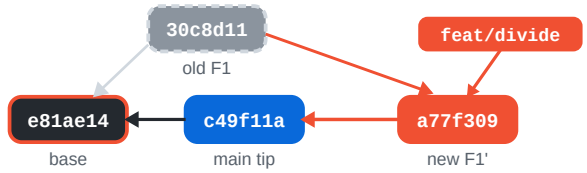
```
$ git reflog
442b5b5 HEAD@{0}: reset: moving to 442b5b5
c49f11a HEAD@{1}: commit: Add multiply method to Calculator
+ Run: git reset --hard c49f11a -> multiply() is 100% restored!
```

06 Rebasing (Standard & Interactive), --force-with-lease & Stashing

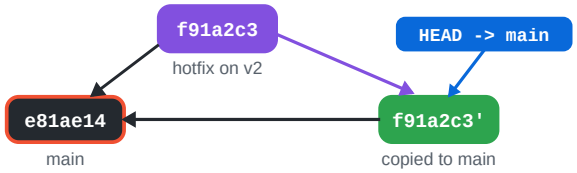
Keeping commit history linear, squashing WIP commits, and saving uncommitted work safely

ILLUSTRATION 5: git rebase main (REPLAYING COMMITS LINEARLY) & git cherry-pick (SURGICAL COPY)

A. git rebase main (F1 -> F1' with brand-new SHA)



B. git cherry-pick f91a2c3 (Copy 1 Hotfix Commit)



COMMAND EXECUTED: `git rebase -i HEAD~3 (change 2nd & 3rd 'pick' to 'squash' / 's')`

BEFORE: 3 Messy WIP Commits on feat/divide Before Opening a PR

```
* 91b2e4c (HEAD -> feat/divide) fix typo in divide
* 64a0f12 oops forgot zero check
* 30c8d11 add divide method
* e81ae14 (main) Merge feat/multiply
```

AFTER: 1 Clean, Atomic Commit Ready for Pull Request Review

```
pick 30c8d11 add divide method
squash 64a0f12 oops forgot zero check
squash 91b2e4c fix typo in divide
+ Result: * a77f309 (HEAD -> feat/divide) Add divide(a, b) with zero-check
```

[WARNING] Three Critical Rebase & Stash Rules Every Software Engineer Must Know

1. **Never Rebase Shared Public main:** Rebasing replaces old commits with new SHA hashes. Only rebase your own feature branches!
2. **Always Use --force-with-lease Instead of --force:** After rebasing a pushed branch, run **`git push --force-with-lease`** (verifies nobody else pushed to the branch before updating it).
3. **The --ours / --theirs Inversion in Rebase:** During **git rebase**, Git checks out **main** first and replays your commits onto it—so **--ours** is **main** and **--theirs** is your feature commit! And when stashing new untracked files, always pass **-u**: **`git stash push -u -m "WIP"`**.

07 Surgical Engineering Tools: cherry-pick, blame, log -S & bisect

Copying a single commit across branches and hunting down production regressions in O(log N) time

Tool & Command	What It Does Under the Hood	Concrete Calculator.java Example
`git cherry-pick <commit-sha>`	Copies the diff of **one specific commit** from another branch and applies it as a new commit on your current branch.	Fixed division-by-zero in commit **f91a2c3** on **feat/v2** and need it on **main** now: `git switch main && git cherry-pick f91a2c3`
`git blame -L 1,10 src/Main.java`	Shows the exact commit SHA, author, and timestamp that last modified each line of a file.	Instantly see whether line 3 of **Main.java** came from commit **e81ae14** or **442b5b5**.
`git log -S "subtract" -p`	**Pickaxe Search:** Searches all history for commits that added or deleted the string **"subtract"**.	Finds commit **7be836b** (where **subtract()** was first introduced) even across 10,000 commits.
`git bisect start` **`git bisect bad HEAD`** **`git bisect good 03391fe`**	**Binary Search Through Time:** Checks out midpoint commits repeatedly until it pinpoints the breaking commit.	If 128 commits happened since **03391fe** and **Calculator.add()** broke, **git bisect** finds the culprit in **7 steps**!
`git commit --amend`	Replaces the tip commit with a new commit combining your staged changes (or updated message).	Forgot to save **Main.java** before committing? Run: `git add src/Main.java && git commit --amend --no-edit`

08 The 'Oh Shit, Git!' Emergency Rescue Playbook & Master Cheat Sheet

Added from Round 2 Multi-Agent Debate — Keep this page handy for real-world emergencies

#	Production Emergency / Symptom	Exact CLI Rescue Sequence
1	"I committed on main instead of creating a new feature branch!"	`# Create branch at current spot, roll main back to cloud, switch to branch:` `git branch feat/my-work && git reset --hard origin/main && git switch feat/my-work`
2	"I ran git reset --hard and destroyed my committed code!"	`# Find the SHA right before the reset (e.g. HEAD@{1}) and restore it:` `git reflog && git reset --hard HEAD@{1}`
3	"I forgot to include a file in my last commit (or made a typo in the message)!"	`git add src/Main.java && git commit --amend -m "Updated message"`
4	"I pushed a buggy commit to shared main and need to undo it safely!"	`# Creates a new commit that reverses the buggy commit (never reset shared main):` `git revert <buggy-commit-sha> && git push`
5	"I started a merge or rebase, hit ugly conflicts, and want to cancel!"	`git merge --abort # or during rebase: git rebase --abort`
6	"I committed a secret file or out/ folder and .gitignore isn't untracking it!"	`git rm -r --cached out/ && git commit -m "Stop tracking out/ directory"`
7	"I checked out an old commit (Detached HEAD) and made new commits there!"	`git switch -c rescued-feature # Saves detached commits onto a real branch`
8	"I accidentally deleted a local branch that had unmerged commits!"	`git reflog && git switch -c recovered-branch <deleted-tip-sha>`
9	"I want to discard all uncommitted local edits in Main.java (or everything)!"	`git restore src/Main.java # Discard unstaged edits in one file` `git restore . && git clean -fd # Discard ALL unstaged edits + untracked files`
10	"Local repo and GitHub repo complain: refusing to merge unrelated histories!"	`git pull origin main --allow-unrelated-histories && git push -u origin main`

● ● ● Master Daily CLI Quick-Reference Card (Setup -> Branch -> PR -> Sync -> Clean)

```
# INITIALIZE & CONNECT TO GITHUB CLOUD (100% CLI)
$ git init && git add . && git commit -m "Initial commit" && git branch -M main
$ gh repo create github-revision --private --source=. --remote=origin --push

# DAILY FEATURE WORKFLOW
$ git switch main && git pull --rebase # Start from latest cloud main
$ git switch -c feat/new-feature # Create & switch to feature branch
$ git status && git diff # Inspect unstaged changes
$ git add . && git diff --staged # Stage & verify snapshot
$ git commit -m "Implement feature" # Save snapshot locally
$ git push -u origin feat/new-feature # Upload branch to GitHub

# PULL REQUEST & CLEANUP
$ gh pr create --fill && gh pr diff && gh pr merge --squash --delete-branch
$ git switch main && git pull && git fetch --prune && git branch -d feat/new-feature
```

[INFO] Your Current Project Status in /Users/youssefmohamed/Projects/Git/Github Revision

Right now in your terminal, **src/Main.java** is paused mid-merge with the live conflict between **main (v1.0)** and **feat/multiply (v2.0)** so you can inspect it whenever you return—or cancel it anytime with **git merge --abort**.

[DEBATE] Multi-Agent Debate (Part 2 — git-staff-critic): Complete 51:48 Crash Course Expansion (Pages 6–7)
To preserve **Pages 1–5 100% untouched** while covering every topic from the 51:48 video (**JN-_7pOscTE**), **git-staff-critic** mandated: (1) Architectural failure analysis of **SCCS/RCS** (file locking) & **CVS/SVN** (central server SPOF) vs **BitKeeper/Git**; (2) Full **.git/** folder breakdown (**hooks/**, **info/exclude**, 256-bucket **objects/** sharding, **refs/heads/**, **refs/tags/**, **HEAD**); (3) 4-level recursive **Commit → Root Tree (040000) → Sub-Tree → Blob (100644)** inspection; (4) Pre-GitHub **Bare Repositories** (**git clone --bare**); and (5) **PAT Auth**, **--allow-unrelated-histories** bridge merges, **Collaborators**, and **Detached HEAD**.

09 VCS Evolution (00:00–05:59) & Full .git/ Sub-Tree Architecture (06:00–18:21)

Why SCCS & SVN failed at scale, unhiding .git in VS Code, and inspecting Root Trees (040000) vs Blobs (100644)

ILLUSTRATION 6: EVOLUTION OF VERSION CONTROL ARCHITECTURES (VIDEO 00:00 – 05:59)

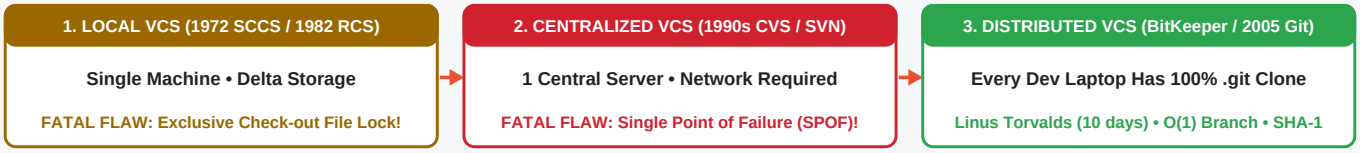
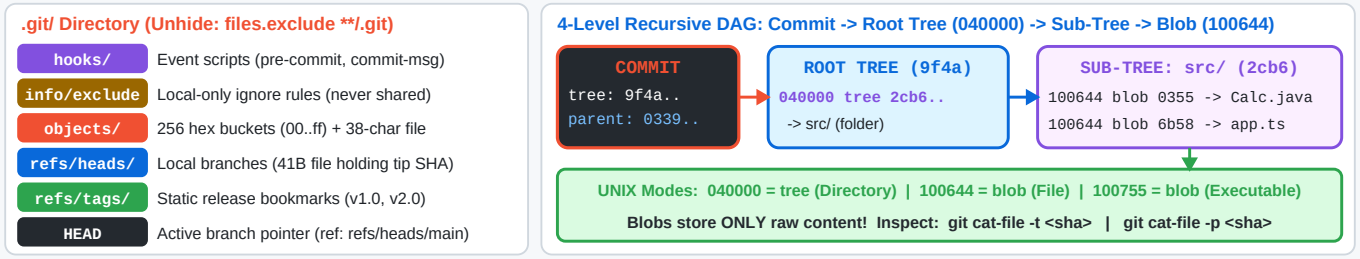


ILLUSTRATION 7: FULL .git/ ANATOMY (06:00–11:30) & 4-LEVEL NESTED SUB-TREE GRAPH (12:20–18:21)



Era & System	Architecture	How It Worked & Why Engineers Needed Git (Video 01:23–05:59)
1972: SCCS (Bell Labs) **1982: RCS**	**Local VCS** (Single Machine)	Built by Marc Rochkind for Unix. Tracked file modifications locally using **exclusive Check-out / Check-in File Locking**: while Dev A edited a file, Dev B was completely locked out!
1990s: CVS **2000: SVN** (Subversion)	**Centralized (CVCS)** (1 Central Server)	Allowed team collaboration, but **100% of history lived on one central server**. Required network for **log/diff/commit**, slow branching, and **Single Point of Failure (SPOF)** if the server disk died.
2000: BitKeeper **2005: Git** (Linus Torvalds)	**Distributed (DVCS)** (Full Local Clone)	Every developer laptop holds a complete clone of **.git/objects** (no SPOF, offline commits). After BitKeeper revoked Linux's free license in 2005, **Linus Torvalds built Git in 10 days**.

COMMAND EXECUTED: `git commit -m "first commit" --> git cat-file -p <commit> / <root-tree> / <sub-tree>`

BEFORE: git add src/app.ts (Creates Only a Raw Blob Object — 12:36)

```
# Unhide .git in VS Code/Cursor (08:08): Settings ->
files.exclude -> delete **/.git
$ git add src/app.ts
# Git hashes content with SHA-1 (40 hex chars):
# First 2 chars = bucket dir (.git/objects/6b/), next
38 = filename
$ git cat-file -t 6b584e --> blob
$ git cat-file -p 6b584e --> console.log('hello'); (NO
filename!)
```

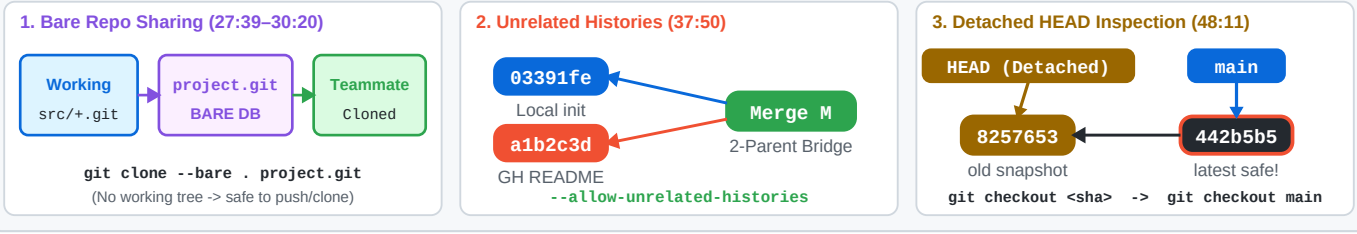
AFTER: Creates Commit -> Root Tree (040000) -> Sub-Tree (src/) -> Blob (100644)

```
$ git cat-file -p <commit-sha> # Points to Root Tree
SHA + Author
$ git cat-file -p <root-tree-sha> # Shows directory
entry (040000 = tree):
+ 040000 tree 2cb657c... src
$ git cat-file -p 2cb657c # Shows file inside src/
(100644 = blob):
+ 100644 blob 6b584e... app.ts
```

10 Bare Repos (27:39), PAT & Unrelated Histories (34:12), Collaborators (39:41) & Detached HEAD

Pre-GitHub sharing (git clone --bare), fixing unrelated histories, PR reviews, and full 51:48 timestamp index

ILLUSTRATION 8: BARE REPOS (27:39) • UNRELATED HISTORIES (37:50) • DETACHED HEAD (48:11)



COMMAND EXECUTED: `git clone --bare . project.git | git pull origin main --allow-unrelated-histories`

BEFORE: Pre-GitHub Bare Repo Sharing (27:39) & Unrelated Histories Trap (37:50)

```
1. Normal Repo: Has working files + .git/. Rejects direct git push!
2. Unrelated Histories Error (37:50):
   - Local repo has Root Commit A (from local 'git init')
   - GitHub repo has Root Commit B (checked 'Add README / .gitignore')
   - 'git pull origin main' fails: fatal: refusing to merge unrelated histories
```

AFTER: Bare Repo Created for Team Clones & Both Root Histories United

```
+ Bare Repo: 'git clone --bare . project.git' holds ONLY .git DB
  Teammates run: git clone project.git project (how GitHub works!)
+ PAT Auth (34:12): https://<PAT_TOKEN>@github.com/<user>/<repo>.git
+ Branch Rename (36:53): git branch -M main
+ Bridge Merge (37:50): Unite Root A + Root B -> save (:wq) -> git push -u origin main
```

Video Segment	Concept / Action	Exact CLI Command / GitHub UI Workflow
39:41 – 44:47	**Invite Collaborator & Push Branch**	Repo **Settings** → **Collaborators** → **Add people** (without this, teammate gets **403 Permission denied**!). Teammate clones repo, runs **`git checkout -b feature/profile`**, commits, and runs **`git push origin feature/profile`**.
44:48 – 48:10	**Pull Request & Code Review**	Open PR (**base: main** ← **compare: feature/profile**). Reviewer opens **Files changed**, clicks **+** on specific lines to add inline comments, clicks **Review changes** → **Approve** → **Submit review** → **Merge pull request**.
48:11 – 49:52	**Inspect Old Commit (Detached HEAD)**	**`git checkout <commit-sha>`** moves **HEAD** directly to an old snapshot without deleting newer commits. Test/inspect old code safely, then return to latest state via **`git checkout main`**.
49:53 – 50:52	**Permanently Delete Newer Commits**	**`git reset --hard <commit-sha>`** rewinds **main** back to <sha> and deletes newer commits locally; **`git push origin main --force`** overwrites GitHub (use **git reflog** to rescue!).

Complete 51:48 Crash Course Index (Part 1: 00:00 – 18:21)

00:00 – Intro & Fast Software Release Challenges (Mod 09)
01:23 – Local VCS: 1972 SCCS, 1982 RCS & File Locking (Mod 09)
03:05 – Centralized VCS: CVS, SVN & Single Point of Failure (Mod 09)
03:42 – Distributed VCS (BitKeeper) & Full Local Clones (Mod 09)
04:14 – Linus Torvalds & The 10-Day Birth of Git (Mod 09)
06:00 – Installing Git, **git init** & Unhiding **.git** in VS Code (Mod 09)
08:36 – Inside **.git/**: **hooks**, **info/exclude**, **objects**, **refs** (Mod 02 & 09)
10:09 – **git status**, **git add** & Local **.git/info/exclude** (Mod 01 & 09)
12:36 – Inspecting **blob** & **tree** Objects (**git cat-file -t/-p**) (Mod 02 & 09)
14:33 – **git commit -m**, Root Tree (040000) vs Sub-Tree (Mod 02 & 09)

Complete 51:48 Crash Course Index (Part 2: 18:22 – 51:49)

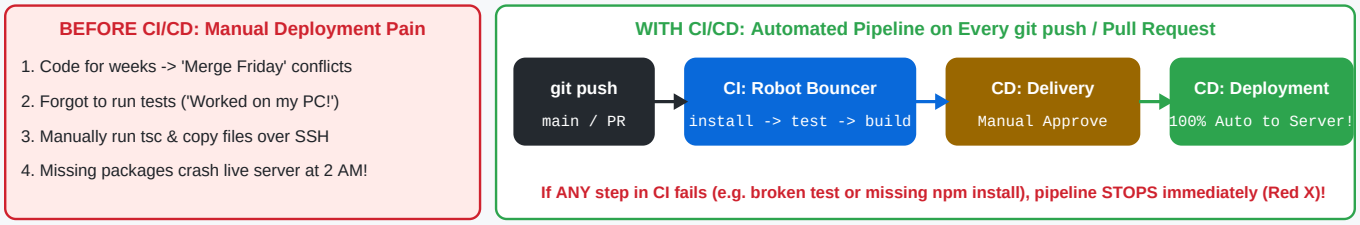
18:22 – Branches: **git branch**, **checkout**, **checkout -b** (Mod 03)
21:21 – Merging Branches (**git merge**) & Fast-Forward (Mod 03)
24:10 – Triggering & Resolving Merge Conflicts Live (Mod 03)
27:39 – Pre-GitHub Sharing: Bare Repos (**git clone --bare**) (Mod 10)
30:21 – GitHub Intro, Creating Repo & **.gitignore** Templates (Mod 02 & 10)
34:12 – GitHub Personal Access Token (PAT) Authentication (Mod 10)
36:53 – **git branch -M main** & **--allow-unrelated-histories** (Mod 08 & 10)
39:41 – Inviting Collaborators & 2nd-Machine Branch Push (Mod 04 & 10)
44:48 – Creating, Reviewing, Approving & Merging a PR (Mod 04 & 10)
48:11 – Detached **HEAD**, **git reset --hard** & CI/CD Intro (Mod 05, 10, 11)

[DEBATE] Multi-Agent Debate (Part 3 — cicd-devops-critic): Beginner-First CI/CD & GitHub Actions (Pages 8–9)
To make CI/CD effortless for a beginner while covering 100% of **Video 2 (M7BZ5e8a1NM)**, **Video 3 (iUrQWj-S2iE)**, and our hands-on demo, **cicd-devops-critic** mandated **5 core mental models**: (1) **The '60-Second Self-Destructing Computer' Rule** (why Runners start empty, why **actions/checkout@v5** is needed, and why Artifacts bridge Job 1 & Job 2); (2) **CI Robot Bouncer vs. CD Robot Courier** (Continuous Delivery vs. Continuous Deployment); (3) **The 3-Keyword YAML Decoder** (**run:** vs **uses:** vs **with:**); (4) **The Padlock [Lock] & Physical Key [Key] SSH Analogy** (**ssh-keygen** vs AWS EC2 **.pem** vs GitHub Secrets); and (5) **The 2 Classic Demo Bugs** (parallel jobs race condition fixed by **needs:**, and duplicate **pm2 start** fixed by **||** fallback).

11 CI/CD Simply Explained (M7BZ5e8a1NM) & GitHub Actions CI Pipeline (iUrQWj-S2iE)

The 'Self-Destructing Computer' mental model, run vs uses vs with, checkout@v5, setup-node@v6 & mock testing

ILLUSTRATION 9: WHY CI/CD IS A NECESSITY (VIDEO M7BZ5e8a1NM) — MANUAL HELL VS. AUTOMATED FACTORY



[IMPORTANT] The #1 Beginner Mental Model: Every Job Rents a '60-Second Self-Destructing Computer' (Runner)
When GitHub runs **runs-on: ubuntu-latest**, it spins up a **brand-new, 100% empty Ubuntu Linux virtual computer**. The second the job finishes, GitHub **destroys that computer and wipes its hard drive forever**!
• **Why do we need `actions/checkout@v5`?** Because that rented computer starts completely empty—your project files aren't on it yet! **checkout@v5** clones your GitHub repo onto that temporary computer so the next steps can see your code.
• **Why do we need `actions/setup-node@v6`?** It installs **Node.js (v24)** and **npm** onto that temporary computer so you can run **npm install**, **npm run test**, and **npm run build**.
• **How to read `actions/checkout@v5`: actions/** = made officially by GitHub | **checkout** = tool name | **@v5** = Version 5.

YAML Keyword	Plain-English Meaning for Beginners	Example from Your .github/workflows/ci-cd.yaml

`on:`	**The Trigger:** Tells GitHub when to run this pipeline automatically.	`on:`
		`  push:`
		`  branches: [main, master, "feature/**"]`
		`  pull_request: # or schedule: - cron: ...`

`uses:`	**Ready-Made Shortcut Plugin:** Downloads an official/community tool from GitHub Marketplace so you don't write complex Linux scripts.	`uses: actions/checkout@v5 # Clones repo code` `uses: actions/setup-node@v6 # Installs Node.js`
`with:`	**Plugin Settings (Inputs):** Passes options into the **uses:** plugin right above it (like function arguments).	`with:` `  node-version: 24 # Passed to setup-node@v6`

`run:`	**Terminal Command:** Types and runs a normal shell command inside the rented Ubuntu virtual machine.	`run: npm install # Installs dependencies`
		`run: npm run test # Runs our test suite`
		`run: npm run build # Compiles TS -> dist/`

COMMAND EXECUTED: `Add src/app.test.ts (assert.strictEqual(1 + 1, 2)) + "test": "tsx --no-warnings --test src/app.test.ts"`

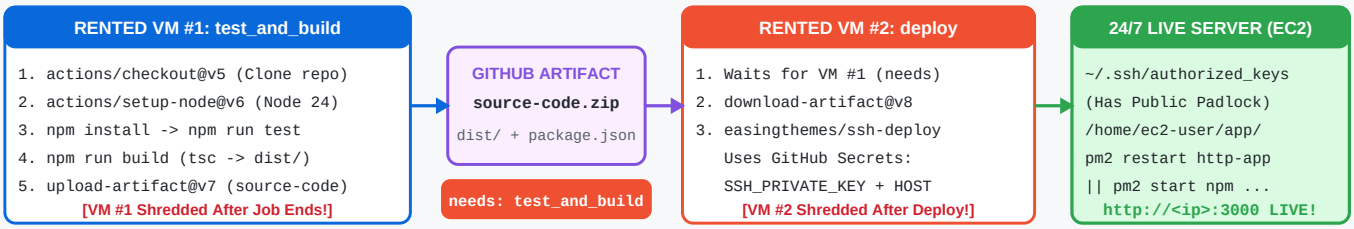
BEFORE: Missing Dependencies or Failing Test Stops the Pipeline (Video 16:40)
In Video (16:40), the instructor removed 'run: npm install':
- 'npm run build' (tsc) immediately crashed with Exit Code 127!
- When ANY step in 'test_and_build' fails, the whole pipeline
  STOPS right there so broken code never reaches your server.

AFTER: CI Gate Passes: Install -> Test -> Build -> Upload Artifact
+ Step 1: npm install (downloads packages into runner VM)
+ Step 2: npm run test -> PASS: mock test that always passes
+ Step 3: npm run build -> compiles src/app.ts into dist/app.js
  (Note: local dist/ is in .gitignore because CI builds it fresh!)

12 Artifacts, needs:, SSH + rsync Deploy, Secrets, pm2 & Hands-On Troubleshooting

How Job 1 passes files to Job 2, the 2 classic video bugs (30:16 & 36:18), SSH keys, and our 5 session fixes

ILLUSTRATION 10: GITHUB ACTIONS 2-VM PIPELINE ARCHITECTURE (VIDEO iUrQWj-S2iE)



COMMAND EXECUTED: `Fix 1: Add 'needs: test_and_build' | Fix 2: pm2 restart http-app || pm2 start npm --name http-app -- start`

BEFORE: The 2 Classic Demo Bugs in Video iUrQWj-S2iE (30:16 & 36:18)

BUG 1 (30:16): Defined 'deploy:' without 'needs: test_and_build'

- GitHub runs all jobs in PARALLEL (at the same time) by default!

- 'deploy' crashed immediately: Artifact 'source-code' not found!

BUG 2 (36:18): Used 'pm2 start dist/server.js' on every deploy

- Spawned duplicate processes on Deploy #2, #3 instead of updating!

AFTER: Sequential Execution + Zero-Downtime Process Restart

+ Fix 1 ('needs: test_and_build'): Forces 'deploy' (VM #2) to WAIT

until 'test_and_build' (VM #1) finishes uploading 'source-code'.

+ Fix 2 ('A || B' in bash = Run A; ONLY if A fails, run B):

Deploy #1: 'pm2 restart' fails -> runs 'pm2 start' (first launch).

Deploy #2+: 'pm2 restart' succeeds -> updates running app!

● ● ● Complete Annotated .github/workflows/ci-cd.yaml (With Beginner Comments & Node 24 @v7/@v8 Upgrade)

```yaml
name: HTTP APP workflow
on: { push: { branches: [main, master, "feature/**"] } } # TRIGGER: Run on push to these branches
jobs:
  test_and_build: # JOB 1 (CI): Rents fresh Ubuntu VM #1
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5 # Clones our repo onto empty VM #1
      - uses: actions/setup-node@v6 # Installs Node.js v24 & npm (with: node-version: 24)
        with: { node-version: 24 }
      - run: npm install && npm run test # Installs packages & runs src/app.test.ts
      - run: npm run build # Compiles TypeScript (src/) -> JavaScript (dist/)
      - uses: actions/upload-artifact@v7 # Saves dist + package.json before VM #1 is destroyed
        with: { name: source-code, path: "dist\npackage.json" }
  deploy: # JOB 2 (CD): Rents brand-new Ubuntu VM #2
    runs-on: ubuntu-latest
    needs: test_and_build # MUST wait for Job 1 to finish!
    steps:
      - uses: actions/download-artifact@v8 # Grabs 'source-code' saved by Job 1 into ./
        with: { name: source-code, path: ./ }
      - uses: easingthemes/ssh-deploy@main # Copies files via rsync over SSH to live server
        with:
          SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }} # Reads hidden key from GitHub Secrets
          SOURCE: "dist/ package.json" # Trailing '/' copies contents of dist/
          REMOTE_HOST: ${{ secrets.REMOTE_HOST }} # Server IP / DNS (+ REMOTE_USER / TARGET)
          SCRIPT_AFTER: cd ${{ secrets.REMOTE_TARGET }} && (pm2 restart http-app || pm2 start npm --name http-app -- start)
```

Hands-On Question / Issue We Solved	Simple Explanation & Exact Fix
1. How did you get `SSH_PRIVATE_KEY`**?** (**ssh-keygen** vs AWS EC2 **.pem**)	Running **`ssh-keygen -m PEM -t rsa -b 4096`** creates a matched pair: (1) **Private Key [The Key]** → pasted into **GitHub Repo** → **Settings** → **Secrets** → **Actions** → **SSH_PRIVATE_KEY**; (2) **Public Key [The Padlock]** → placed in the server's `~/.ssh/authorized_keys`. On **AWS EC2**, AWS bolts the Padlock onto the server automatically and downloads the `.pem` Private Key to your computer.
2. Why `403 Permission denied` **on** `git push`**?** Do I need to remake my commits?	Cloning the instructor's repo pointed `origin` to `alaamohamed58/application-server`. Your local commits are **already saved on your Mac**—you do not need to re-commit! Just point `origin` to your own repo (**`git remote set-url origin https://github.com/y0ussefmohamed/github-actions-ci-cd.git`**) and run **`git push`**.
3. How to fix the `Node.js 20 is deprecated` **warning on** `upload-artifact@v4`**?**	`actions/upload-artifact@v4` was built for Node 20. Upgrading to **`actions/upload-artifact@v7`** and **`actions/download-artifact@v8`** runs natively on Node 24 and removes the warning completely.
4. Where do I see the deployed website in my browser?	In the video, the instructor rented a 24/7 **AWS EC2 server** with a public IP (`http://<ec2-ip>:3000`). Without a 24/7 external server, deploying inside GitHub's temporary runner VM shuts down as soon as the `deploy` job ends.